

MACHINE LEARNING • UNSUPERVISED

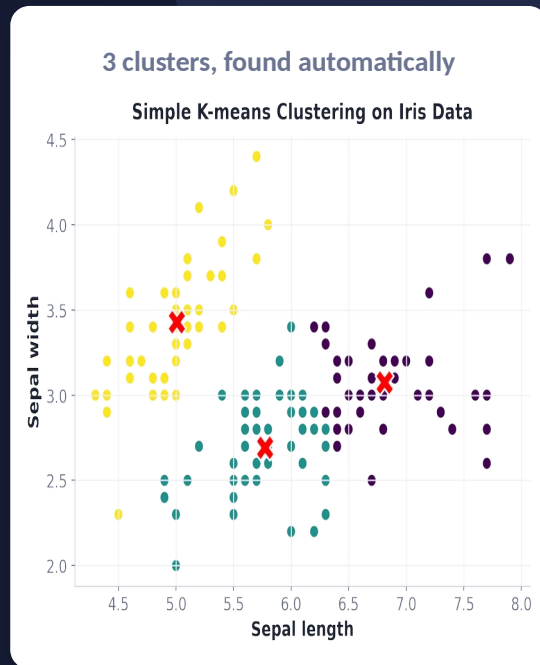
K-Means Clustering

Grouping **Iris flowers** into clusters — **no labels needed**, just patterns.

BUILT WITH



Python · matplotlib · scikit-learn



Clustering: learning without labels

Most models we've seen are **supervised** — they learn from labelled examples. Clustering is **unsupervised**: it gets only the data and must find the groups itself.



No labels given

It receives only the inputs X — there is no answer key to copy.



Find natural groups

It gathers points that sit close together in feature space.



Discover structure

Handy when you don't yet know what categories live in your data.

SUPERVISED vs UNSUPERVISED

Supervised

learns from $X \rightarrow Y$ (labels)

Unsupervised

learns from X alone

What is K-Means?

K-Means looks for **k cluster centres** (called centroids). Every point joins the **nearest centroid**, and that grouping is the cluster it belongs to.



k centroids

You choose k up front — here $k = 3$, so the data is split into 3 groups.



Nearest wins

Each point is assigned to whichever centroid it is closest to.



Centres = averages

A centroid sits at the mean position of all the points in its cluster.

THIS MODEL

k = 3

centroids



one centre per cluster

How K-Means works — a repeating loop

1

Place k centroids

Start with 3 centres at random positions.

2

Assign each point

Send every point to its nearest centroid.

3

Move the centroids

Shift each centre to the mean of its points.

4

Repeat

Re-assign and re-centre until nothing changes.



Assign → re-centre → repeat

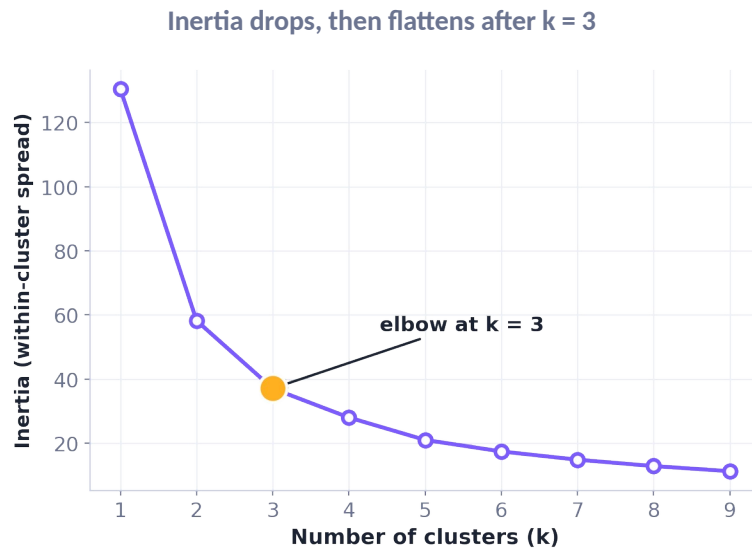
*Stops when the centroids stop moving
— the clusters have settled
(convergence).*

Choosing k — the elbow method

How many clusters? Plot the **inertia** (how tightly points sit around their centre) for each k. It always drops, but the **'elbow'** — where the gain flattens — is a good choice.



For Iris, the elbow is at $k = 3$ — which also matches its 3 real species.



The data K-Means sees

150

points

2

features

0

labels

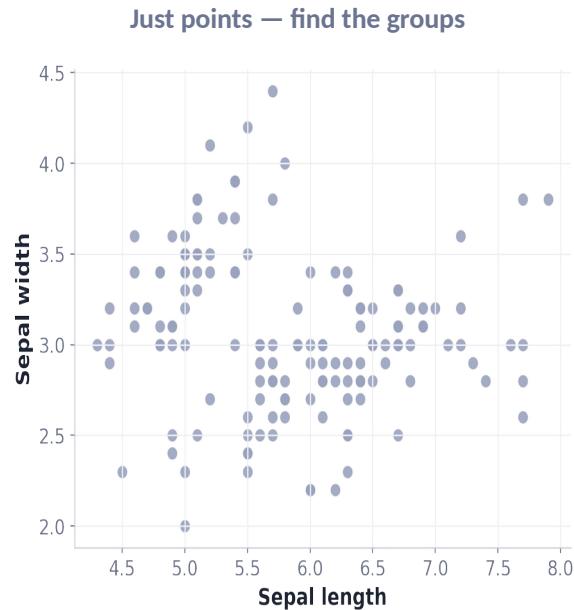
THE TWO FEATURES USED (`X = iris.data[:, :2]`)

Sepal length

Sepal width



No species labels. Iris has 3 species, but K-Means never sees them — it must find groups on its own.



THE WORKFLOW

5 steps from points to clusters



STEP 1

Import



STEP 2

Load data



STEP 3

Fit K-Means



STEP 4

Assign



STEP 5

Plot

Next: every step explained line by line →



STEP 1 • CODE

Import the tools

imports.py



```
# the tools we need  
import matplotlib.pyplot as plt  
from sklearn.cluster import KMeans  
from sklearn.datasets import load_iris
```

LINE BY LINE

- **matplotlib.pyplot**
to draw the cluster scatter.
- **KMeans**
the clustering algorithm itself.
- **load_iris**
the sample flower dataset.



STEP 2 • CODE

Load & pick 2 features

load.py



```
# load iris, keep 2 features
iris = load_iris()

# X = sepal length & sepal width
X = iris.data[:, :2]
```

LINE BY LINE

- **load_iris()**
loads the 150 flowers.
- **[:, :2]**
keep only the first 2 columns: sepal length & width.
- **no Y**
labels are ignored — this is unsupervised.



STEP 3 • CODE

Create & fit K-Means

fit.py



```
# 3 clusters, fixed seed
kmeans = KMeans(n_clusters=3,
                 random_state=42)

# learn the 3 cluster centres
kmeans.fit(X)
```

LINE BY LINE

- **n_clusters=3**
look for 3 groups in the data.
- **random_state=42**
makes the result the same each run.
- **.fit(X)**
runs the loop to find the 3 centroids.



STEP 4 • CODE

Assign clusters & read centres

assign.py



```
# which cluster each point is in  
y_kmeans = kmeans.predict(X)  
  
# the 3 cluster centres (x, y)  
centers = kmeans.cluster_centers_
```

LINE BY LINE

- **.predict(X)**
labels every point 0 / 1 / 2 by its nearest centroid.
- **.cluster_centers_**
the (x, y) coordinates of the 3 centroids.



STEP 5 • CODE

Plot the clusters

plot.py



```
# colour points by cluster
plt.scatter(X[:,0], X[:,1],
            c=y_kmeans, cmap='viridis')

# mark the centres with red X
plt.scatter(centers[:,0],
            centers[:,1], c='red',
            s=200, marker='X')
plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.show()
```

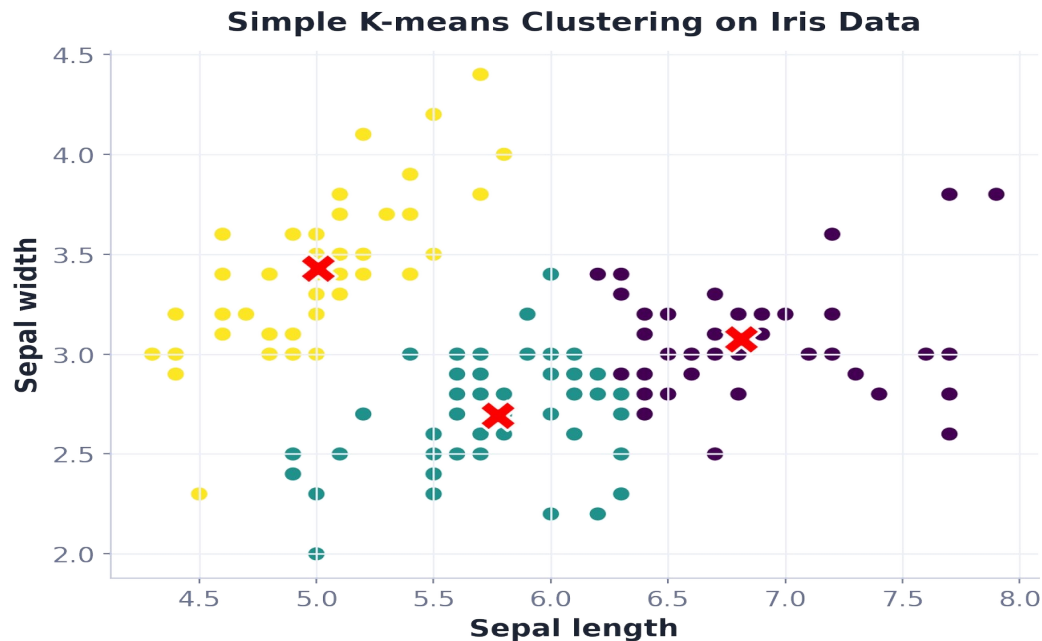
LINE BY LINE

- **scatter(c=y_kmeans)**
colour each point by its cluster (viridis).
- **scatter(marker='X')**
mark the 3 centroids with a red X.
- **plt.show()**
display the finished plot (shown next →).



STEP 5 • OUTPUT

The result of plt.show()



WHAT YOU SEE



3 colours

each is one cluster found by K-Means.



Red X marks

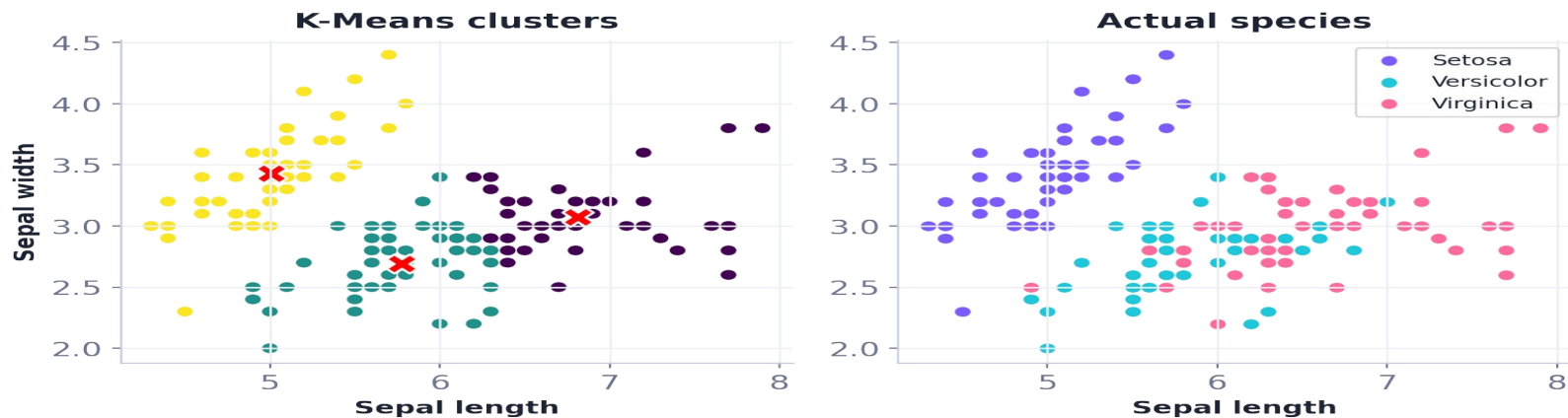
the centroid (centre) of each cluster.



Sizes 47•53•50

the points split fairly evenly.

Did it rediscover the species?



Close, not exact. With only 2 sepal features, K-Means cleanly isolates one species but blends the other two — more features would sharpen the split.

RECAP

What we built



Grouped without labels

K-Means split the flowers into 3 clusters using only sepal length & width.



Assign & re-centre, repeat

It loops between assigning points and moving centroids until they settle.



Centroids mark the centres

The 3 red X's are the cluster centres; a new point joins the nearest one.

k = 3

clusters found

Unsupervised · Iris

